

Filters in MapReduce

Mahmoud Parsian

Ph.D. in Computer Science

What Is a Filter?

A **filter** is a Boolean condition/predicate that limits which `(key, value)` pairs get emitted from a mapper or a reducer.

Examples:

- Drop employee records if `emp_id` is `NULL`
- Drop records containing bad language
- Drop a DNA sequence if its chromosome is undefined
- Drop messages longer than 800 characters

Where Can a Filter Live?

- **In the mapper** — to drop records before they're ever shuffled.
- **In the reducer** — to filter the reducer's *output*, after aggregation.

Both are valid places to filter — the question is **which one is correct** for a given filter, and this document is about answering that question.

The Rule That Decides Where

If the filter depends on aggregated values, it belongs in `reduce()`. Otherwise, it belongs in `map()`.

Putting a filter in the mapper — whenever possible — means fewer `(key, value)` pairs cross the network during Sort & Shuffle. That's the same efficiency motivation behind combiners (see [07_combiners_in_mapreduce.md](#)): ship less, as early as possible.

Mapper Filter: Reject by Word

```
def map(key, value):  
    words = value.split(" ")  
    for word in words:  
        if word.startswith("E"):    # skip words starting with "E"  
            continue  
        emit(word, 1)
```

Correct in the mapper — deciding whether to keep a word never needs to know its eventual total count.

Mapper Filter: Reject by Word Length

```
def map(key, value):  
    words = value.split(" ")  
    for word in words:  
        if len(word) > 2:                # drop words < 3 chars  
            emit(word, 1)
```

Input "a fox of jumped over red fox and jumped" → "a" and "of" are dropped; everything else is emitted as (word, 1). Still no aggregated value involved — mapper is correct.

Mapper Filter: Reject by Record Length

```
def map(key, value):  
    if len(value) < 80:                # drop the whole record  
        return  
    words = value.split(" ")  
    for word in words:  
        if len(word) > 2:  
            emit(word, 1)
```

This filter needs the **entire input record** — something only the mapper ever sees. (More on why this matters in a moment.)

Reducer Filter: Reject by Aggregated Count

```
def reduce(key, values):  
    count = 0  
    for v in values:  
        count += v  
    if count >= 5:                                # needs the FINAL sum  
        emit(key, count)
```

Input (fox, [1,1,1,1,1,1,1]) → count = 7 → kept. A word with count = 3 is dropped. This filter is only decidable **after** summing every value for the key — it cannot run in the mapper, which never sees more than one record at a time.

A Common Mistake: The Wrong Filter in the Reducer

```
def reduce(key, values):  
    # WARNING: not a proper reducer filter – should be in the mapper!  
    if len(key) <= 2:  
        return  
    count = 0  
    for v in values:  
        count += v  
    if count < 5:  
        return  
    emit(key, count)
```

The `len(key) <= 2` check doesn't depend on `values` at all — it could have dropped short words in the *mapper*, before Sort & Shuffle ever shipped them across the network. Putting it in the reducer isn't *wrong* (the output is identical), but it's wasted shuffle traffic.

An Impossible Filter: What the Reducer Can Never Do

```
def reduce(key, values):  
    # We CANNOT implement "drop records shorter than 80 chars" here –  
    # the reducer never sees the original record, only (key, values).  
    ...
```

The "drop the whole record if it's under 80 characters" filter from a few slides ago **cannot** be written in `reduce()` at all — by the time a reducer runs, the original record is gone; all it has is a key and a list of already-mapped values. Some filters aren't just *better* in the mapper — they're only *possible* there.

Try It Yourself: Where Does Each Filter Go?

For each filter below: `map()`, `reduce()`, or impossible in `reduce()`? (Answer on the next slide.)

1. Drop any record shorter than 10 characters.
2. Drop any word whose total frequency is less than 3.
3. Drop any word that appeared in ALL CAPS anywhere in the original record.

Try It Yourself: Answers

1. `map()` — needs the whole original record; no aggregation.
2. `reduce()` — "total frequency" only exists after summing every value for that key.
3. **Only possible in** `map()` — by the time a reducer runs, the original casing/record is gone; the reducer only ever sees `(key, values)`, the same reason the record-length filter earlier couldn't move to `reduce()` either.

Summary: MapReduce Filters

- A filter belongs in `map()` if it does *not* depend on aggregated values — including any filter that needs the whole original record, since only the mapper ever sees that.
- A filter belongs in `reduce()` if it *does* depend on aggregated values (a sum, a count, ...).
- Put every filter as early as it can correctly go — it minimizes the number of `(key, value)` pairs shipped across the network.